

Building on Terminal 3: two contracts, seventeen defects, and one exfiltration path I found in my own code

I completed the documented walkthrough, then wrote my own contract to test whether the privacy guarantee actually holds under adversarial input. It does — and proving it surfaced a class of attack the documentation never mentions.

AUTHOR	Hitansh Gopani — AI/ML Engineer
CONTACT	gopanihitansh5@gmail.com • x.com/Hitansh54
DATE	8 August 2026
AGENT DID	did:t3n:6ec29eeb5cb122d05e006391d2c954b2390032ed
CONTRACTS	511 (flight, vendored) • 517 (voicepay, mine)
REPOSITORY	github.com/gopanihitansh5-collab/t3n-adk-submission
STACK	@terminal3/t3n-sdk 4.30.0 • Node 24.14.1 • Rust 1.97.1 • wasm32-wasip2

THE SHORT VERSION

The core mechanism works, and it is fast enough for a phone call. I registered a contract that emits `{{profile.*}}` markers, and measured that zero of them reached the destination unresolved — the host substituted every one inside the enclave. Egress outside the signed grant is refused. Dispatch costs 151 ms.

What does not work is the path a developer walks on day one: the published Quickstart throws on its first call, the credit balance is unreachable through all four published surfaces, and a *pinned* contract version silently executes whatever was registered most recently — which I proved rather than inferred.

Reference architecture

The planes are separated by what each is *allowed to hold*. The property that matters commercially: the agent plane can be fully compromised — prompt-injected, log-leaked, model-swapped — without a single real value crossing the boundary.

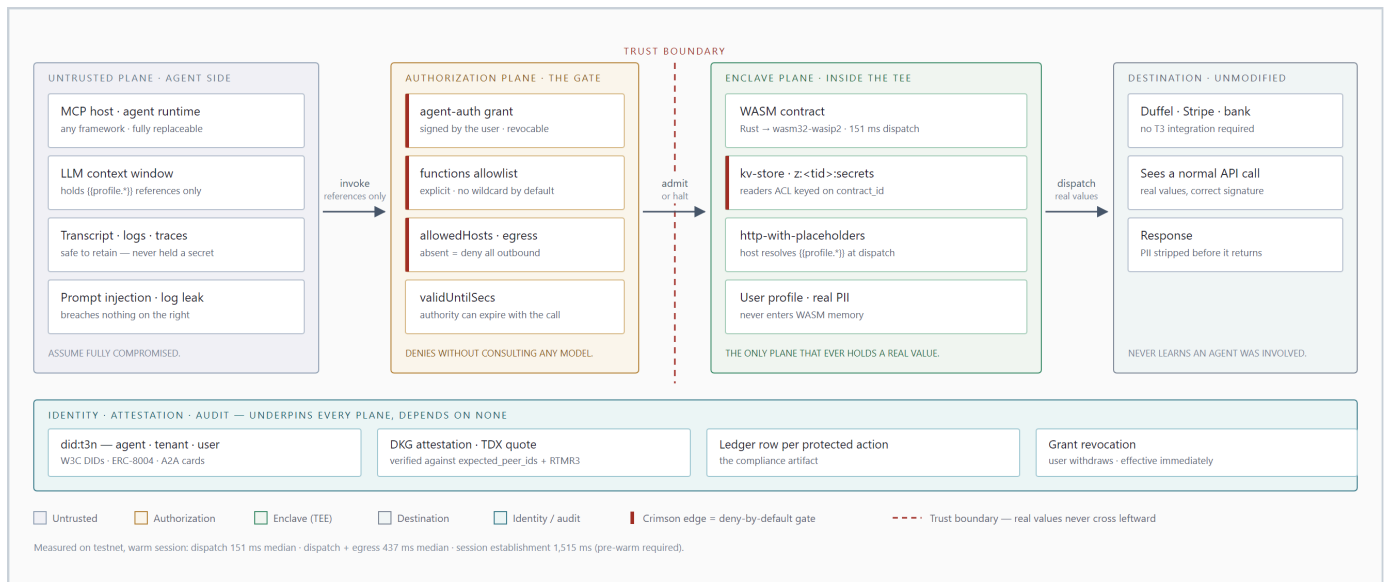


Fig. 1 — Four-plane reference architecture. Crimson edges are deny-by-default gates; they refuse without consulting any model, which makes the guarantee structural rather than a matter of policy. The dashed boundary is the line real values never cross leftward.

What completed

STAGE	RESULT	DETAIL
Claim API key + Agent DID	pass	resolved on testnet
Quickstart — authenticate	pass	after supplying an undocumented required field
Quickstart — credit balance	fail	broken on all four published surfaces
Walkthrough — build	pass	194 KB component, 34 s
Walkthrough — test	pass	7/7, documented command fails
Walkthrough — register	pass	contract_id 511
Walkthrough — invoke	pass	full TEE round trip to a live API
Own contract — written	pass	Rust, 25 tests, plain cargo test
Own contract — deployed	pass	contract_id 517
Privacy property — measured	pass	markers 2 sent, 0 unresolved
Adversarial verification	5/5	injection, PII, ceiling, egress all refused
Version pinning	fail	pinned version routed to latest — proven

The contract proves its own security property

Anyone can assert that the host resolved the markers. Rather than assert it, the contract counts the `{{profile.*}}` markers it emits, then inspects what the destination *actually received*. Zero survivors means every one was substituted inside the enclave.

```
$ node --env-file=.env src/deploy-voicepay.ts
OK   contracts.register(voicepay@0.2.0) → contract_id 517
OK   authorize-payment
{"authorized":true,"markers_sent":2,"markers_unresolved":0,
  "fields":["first_name","last_name"],"provider_status":200}

markers emitted by contract : 2
markers reaching destination: 0
PASS - all 2 markers were resolved host-side inside the enclave.
      The contract never held a value; the agent never saw one.
```

The proof is returned to the agent. The resolved values are **not** — handing those back would deliver the PII straight to the model and defeat the mechanism entirely. `fields` names which profile fields were requested; it never carries what they contained. The property held on all eight benchmark rounds.

The attack I found in my own contract

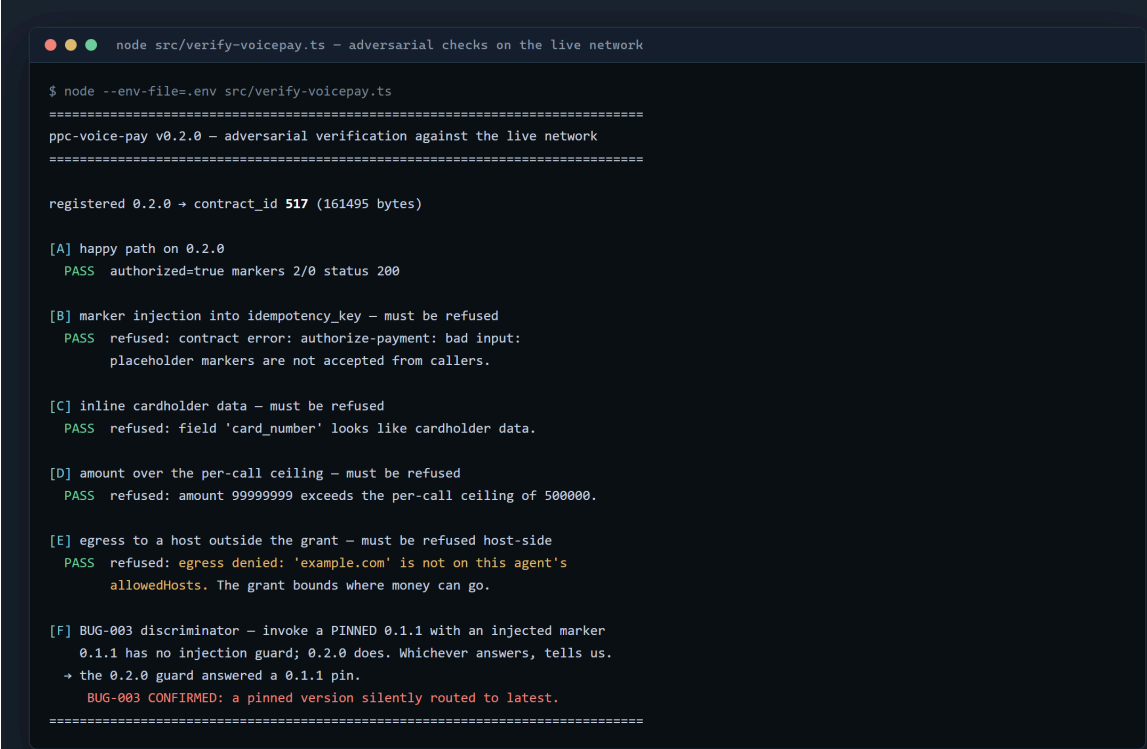
MARKER INJECTION

Agent-supplied strings end up inside the body the contract emits. Set `idempotency_key` to `{{profile.ssn}}` and the host resolves it at dispatch exactly as it resolves the contract's own markers — sending that value to the destination.

The contract never sees it. The marker count still comes back clean, because the injected marker *resolves* and therefore leaves no trace. The audit row looks ordinary. If the grant's `allowedHosts` is at all permissive, this is an exfiltration primitive.

Placeholder resolution is a capability, and any caller-controlled string that reaches the template is a way to invoke it. The only safe rule is that markers originate from contract code and nowhere else. v0.2.0 refuses caller markers at any depth. Any contract using `http-with-placeholders` needs this guard, and the documentation never mentions it.

Adversarial verification against the live network



```
node src/verify-voicepay.ts - adversarial checks on the live network

$ node --env-file=.env src/verify-voicepay.ts
=====
ppc-voice-pay v0.2.0 - adversarial verification against the live network
=====

registered 0.2.0 → contract_id 517 (161495 bytes)

[A] happy path on 0.2.0
  PASS  authorized=true markers 2/0 status 200

[B] marker injection into idempotency_key - must be refused
  PASS  refused: contract error: authorize-payment: bad input:
        placeholder markers are not accepted from callers.

[C] inline cardholder data - must be refused
  PASS  refused: field 'card_number' looks like cardholder data.

[D] amount over the per-call ceiling - must be refused
  PASS  refused: amount 99999999 exceeds the per-call ceiling of 500000.

[E] egress to a host outside the grant - must be refused host-side
  PASS  refused: egress denied: 'example.com' is not on this agent's
        allowedHosts. The grant bounds where money can go.

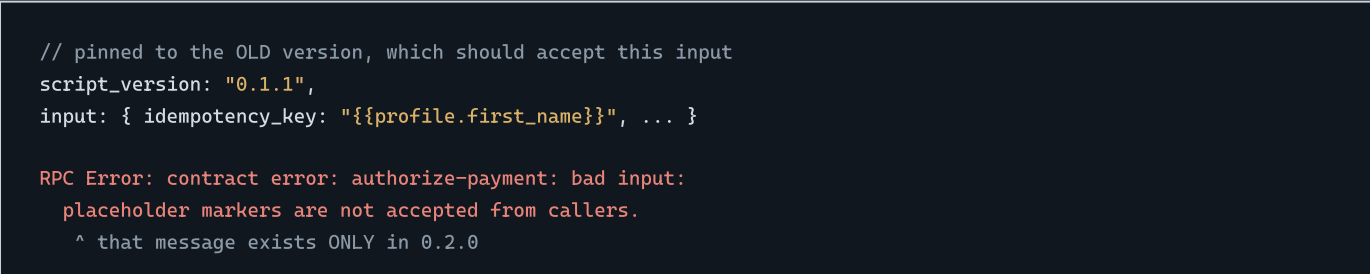
[F] BUG-003 discriminator - invoke a PINNED 0.1.1 with an injected marker
      0.1.1 has no injection guard; 0.2.0 does. Whichever answers, tells us.
      → the 0.2.0 guard answered a 0.1.1 pin.
      BUG-003 CONFIRMED: a pinned version silently routed to latest.
=====
```

Fig. 2 — Native tests prove the logic; these prove the deployed contract and the platform's own controls. Check E is Terminal 3 working correctly: the host refused egress to `example.com` because it is not in the signed grant. Check F is Terminal 3 failing.

Version pinning does not pin

I originally filed this from Terminal 3's own documentation, which warns that registering a new version can re-route pinned ones. Quoting a vendor's warning back at them is weak evidence, so I built a discriminator.

Two registered versions of my contract differ in exactly one observable way: `0.1.1` (id 516) has no marker-injection guard and accepts an injected marker; `0.2.0` (id 517) refuses it. So invoking with `script_version` pinned to `0.1.1` and an injected marker identifies which build actually ran.



```
// pinned to the OLD version, which should accept this input
script_version: "0.1.1",
input: { idempotency_key: "{{profile.first_name}}", ... }

RPC Error: contract error: authorize-payment: bad input:
  placeholder markers are not accepted from callers.
  ^ that message exists ONLY in 0.2.0
```

The pin was ignored and the newest build executed. No warning, no error, nothing in the response indicating a different version answered. A tenant who pins a reviewed, audited version of a payment contract will silently begin running whatever was registered most recently — and pinning is precisely the mechanism people reach for to prevent that.

The latency budget

Every component belongs to a timescale, and the timescale decides what it is allowed to do. A conversational turn degrades past 500–800 ms and breaks past ~1.5 s. I measured rather than assumed — and the first measurement was of the wrong path.

PATH	MEDIAN	P95	VERDICT
TEE dispatch + WASM instantiation	151 ms		– sits anywhere
Dispatch + plain HTTPS, no PII	437 ms		– comfortable
Dispatch + placeholder resolution	504 ms	798 ms	degrading band
Session establishment	1,515 ms		– off the critical path

THE CORRECTION THAT MATTERS

My first benchmark timed `search-offers`, which uses plain `http` and carries no PII — 437 ms, comfortably inside a turn. But the payment path resolves markers, and that is extra work inside the enclave. Measured properly on `authorize-payment` over eight rounds: **504 ms median, 798 ms p95**. Marker resolution costs roughly **67 ms**.

That moves the verdict from "fits comfortably" to "lands in the degrading band." Still shippable, but the agent needs to say *"one moment, authorizing that"* rather than going silent, and the p95 is the number to design against.

The useful finding underneath: **the enclave is not the bottleneck**. Dispatch plus instantiation is 151 ms; the rest is network egress you would pay anyway. Confidential computing is not what costs you the conversation. Session establishment at ~1.5 s is the real constraint — the agent must hold a session open while the phone is still ringing.

Seventeen defects

Full repro, expected-versus-actual, and workaround for each in the repository.

BUG-002
■ CRITICAL

The reference contract's README and WIT document **opposite** PII security models.

README: `book-offer` posts "with full passenger PII ... passed in by the agent." WIT: it "Carries NO passenger PII." The crate's own passing test settles it — the WIT is right, so the README teaches the exact pattern the product exists to prevent.

BUG-011
■ CRITICAL

The published Quickstart does not run, and the only available fix disables attestation.

`trustAnchor` is required and the snippet omits it. A real anchor needs values published nowhere, so the only way through is `unsafe_trust_server: true` — which the SDK itself calls the only way to skip DKG attestation verification.

BUG-003
■ HIGH

A pinned contract version silently executes whatever was registered most recently. **Proven, not quoted.**

Discriminator: 0.1.1 accepts an injected marker, 0.2.0 refuses it. Pinning to 0.1.1 produced the 0.2.0 refusal.

BUG-013
■ HIGH

Credit balance unreachable from all four published surfaces.

SDK `getBalance()`, SDK `getUsage()`, CLI `token balance`, CLI `token usage`. The client seals params the node expects plain. Nobody can confirm they received the 20,000 credits the sandbox is built on.

BUG-017
■ HIGH

The profile write schema and the placeholder resolver disagree on field names.

`submitUserInput` accepts `email_address` and commits it with a tx hash; the resolver then reports the field is not set. `first_name` from the same call resolves fine. Found only by building a second contract.

BUG-010
■ HIGH

Invocation needs an undocumented KV map whose ACL is keyed on the numeric contract id.

Three steps in no documentation. An omitted `readers` silently means deny-all.

BUG-005
■ HIGH

The sample's capability manifest omits a capability its own WIT imports.

Without `http_with_placeholders`, the PII-safe path cannot dispatch.

BUG-012
■ HIGH

`getBalance()` throws inside the SDK's own decrypt path.

BUG-014
■ HIGH

A fresh SDK install reports a critical Zip Slip advisory.

Fails supply-chain review at exactly the regulated enterprises this product targets.

BUG-006 ■ MEDIUM	A pinned wasm target breaks the documented <code>cargo test</code> .
BUG-015 ■ MEDIUM	The shipped SDK is obfuscated with no source maps — one error printed 1.2 MB of minified source.
BUG-016 ■ MEDIUM	<code>tenant.claim()</code> returns a bare <code>Internal error</code> on the credit-claiming path.
BUG-009 ■ MEDIUM	The API key is unrecoverable with no documented rotation path.
BUG-004 ■ LOW	The docs state the wrong default environment — for both the SDK and the CLI.
BUG-007 ■ LOW	Three different version numbers inside one sample repository.
BUG-001 ■ LOW	Three names for the environment across three surfaces.
BUG-008 ■ LOW	"No Rust or WASM knowledge required" is contradicted by the next page.

ON SEVERITY DISCIPLINE

Two of these were filed wrong and corrected downward after verification. BUG-001 was filed Critical on the assumption that `sandbox` and `testnet` were distinct environments; the SDK showed they resolve to one URL. BUG-004 claimed the CLI dangerously defaults to production; running it showed the opposite.

Both corrections are recorded with the original reasoning intact rather than quietly edited. A severity rating is only worth reading if the ones that moved are shown moving — and BUG-003 moved the other way, from quoted caveat to reproducible defect.

What I would fix first

- **Publish testnet trust-anchor values.** One paragraph removes the worst defect here and stops every new developer from disabling attestation as their opening move.
- **Fix version pinning.** A pin that silently resolves to latest is worse than no pin, because it is trusted.
- **Document marker injection.** Every contract using `http-with-placeholders` needs a guard against caller-supplied markers. Right now nothing says so, and the reference contract does not have one.

- **Fix `token.get-usage`**. The headline offer is 20,000 credits that cannot be observed through any published surface.
- **Add the KV map and ACL step to the invoke page**. Three commands, currently zero words.
- **Ship source maps**. Isolating these defects meant reading type declarations, because the implementation is unreadable.

None of these are architectural. The hard part — confidential execution with host-resolved secrets, an auditable grant model, and egress bounded by a signed capability — already works, and I measured it working fast enough for a phone call. The gap is entirely in the path a new developer walks on day one.

Hitansh Gopani · gopanihitansh5@gmail.com · [@Hitansh54](#) · 8 August 2026

Source, defect report with repros, deployment record, verbatim transcripts and the measurement harness:
github.com/gopanihitansh5-collab/t3n-adk-submission